

The Sprout Conversational AI System

Architecture and Algorithms for Real-Time Multimodal Learner Interaction,
Information Recording, and Adaptive Dialogue

Nguyen Thanh Minh Duc

Student ID: 2470734

Email: ntmduc.sdh242@hcmut.edu.vn

Zalo: 0931416201

Masters Programme

HCMC University of Technology

April 29, 2026

Abstract

Sprout is an adaptive learning platform built around a real-time multimodal conversational tutor (voice, text, webcam, and screen sharing). This report describes the production AI subsystem that powers those interactions: a JWT-validated *Gemini Proxy* WebSocket gateway that connects browser clients to Google Gemini Live API, forwards bidirectional media/messages, and tracks session/token telemetry. Replacing the previous LiveKit/WebRTC path reduces architectural complexity and supports low-latency dialogue targets (p95 round-trip < 500 ms). The platform records interaction state in a compact Firestore schema (`users`, `courses`, `lessons`, `conversations`, `messages`, `gemini_sessions`) to enable context priming, completion detection, feedback-driven adaptation, and parent-facing progress summaries. We formalize six core algorithms: authenticated session establishment, tokenized routing, asynchronous knowledge-map generation (Cloud Tasks), dual-channel lesson completion reconciliation, trigger-based conversation feedback summarization, and anonymous-to-authenticated progress linking via HTTP-only cookie identity. We also document deployment on Cloud Run, a typed TypeScript SDK, and the observability stack (Cloud Logging, Mixpanel, AgentOps). Finally, we outline the next architecture step: function-calling with coordinator/specialist multi-agent orchestration (Visual, Navigation, Assessment, and Accessibility agents) on top of the same gateway and data foundations.

Keywords: multimodal LLM; Gemini Live API; WebSocket gateway; JWT authentication; conversational tutor; knowledge map generation; adaptive learning; function calling; multi-agent systems; Firestore.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Problem Statement	4
1.3	Engineering Questions and Objectives	5
1.4	Contributions	5
1.5	Report Organisation	6
2	Literature Review	6
2.1	Conversational Intelligent Tutoring Systems	6
2.2	Multimodal Large Language Models and the Gemini Live API	7
2.3	Real-Time Media Gateways: WebRTC and WebSocket	7
2.4	Function Calling and Tool-Augmented Agents	7
2.5	Educational Knowledge Graphs	7
2.6	Implicit-Feedback Session Analytics	8
2.7	Approximate Nearest-Neighbour Retrieval	8
3	System Context and Use Cases	8
3.1	Stakeholders and Roles	8
3.2	Multimodal Usage Scenarios	9
3.3	Core User Flow	9
3.4	Recorded Data Assets	10
3.5	Privacy and Recording Boundaries	10
4	Methodology	11
4.1	System Topology	11
4.2	Algorithm A1: JWT-Validated Session Establishment	12
4.3	Algorithm A2: Tokenised Message Routing	12
4.4	Algorithm A3: Asynchronous Knowledge-Map Generation	13
4.5	Algorithm A4: Dual-Channel Lesson Completion Detection	14
4.6	Algorithm A5: Conversation Feedback Processing	14
4.7	Algorithm A6: Anonymous-to-Authenticated Progress Linking	15
4.8	Multimodal Session Orchestration	16
5	Implementation Details	17
5.1	Repository Layout	17
5.2	Gemini Proxy Server	17
5.3	Client SDK	18
5.4	Backend Service Surface	20
5.5	Cloud Run Deployment	21
5.6	Configuration and Secrets	22
5.7	Frontend Components	22

6	Evaluation Methodology	22
6.1	Operational Metrics	22
6.2	Behavioural Metrics	22
6.3	Cost Metrics	23
6.4	Algorithm-Level Validation	23
6.5	Observability Stack	24
6.6	Alerting	24
6.7	Load Testing	25
6.8	Comparative Baseline	25
7	Discussion	25
7.1	What This Work Achieves	25
7.2	Limitations	26
7.2.1	Single-Region Proxy Footprint	26
7.2.2	Single-Vector User Model	26
7.2.3	External AI Dependency	26
7.2.4	Offline Evaluation Only	27
7.3	Threats to Validity	27
8	Operations and Reproducibility	27
8.1	Environment Separation	27
8.2	Terraform State	28
8.3	Secret Management	28
8.4	Health Checks and Synthetic Tests	28
8.5	Alerting	28
8.6	Rollback	28
8.7	Reproducing the Pipeline	29
8.8	Determinism Caveats	29
8.9	Ethics and Compliance	30
9	Conclusion and Future Work	30
9.1	Conclusion	30
9.2	Function Calling	30
9.3	Multi-Agent Coordination	31
9.4	Calibrated Lesson Recommendation	32
9.5	On-Device Privacy Processing	32
9.6	Closing Remarks	33

1 Introduction

1.1 Background and Motivation

Adaptive learning platforms remain dominated by static course graphs and pre-recorded lectures. While such material scales economically, it cannot react to the learner mid-sentence: it cannot recognise that a user mispronounced a word, that their gaze has drifted from the camera, or that a piece of code on the shared screen contains a bug. The recent generation of multimodal large language models (LLMs)—in particular Google’s Gemini Live API, which streams audio, images and text in a single bidirectional channel —has changed this calculus. A single inference endpoint can now hold a spoken dialogue, watch the learner via webcam, look at the screen the learner shares, and reply with synthetic speech, all within latency budgets acceptable for tutoring ($\sim 300\text{--}700$ ms one-way).

Sprout is built on this premise. It is an AI-powered adaptive learning platform that personalises skill acquisition for adult learners, neurodiverse learners (with attention or autism-spectrum profiles), and vocational students whose practice naturally involves doing, demonstrating and being seen. The product surfaces three user-visible behaviours: **natural-language skill discovery** (the user types or speaks what they want to learn), **visual knowledge maps** (an AI generates a prerequisite graph of lessons in the React Flow canvas), and a **multi-modal teaching assistant** (the learner enters a session and talks, types, shows their face or shares their screen).

What makes those behaviours economically interesting is replacement pressure. Human one-on-one tutoring costs roughly \$20–\$50/hour; an LLM-driven session approaches \$2–\$3/hour at typical token usage. The technical question—*how do we deliver such a session reliably, cheaply and observably?*—is the subject of this report.

1.2 Problem Statement

The Sprout AI subsystem must satisfy four objectives concurrently:

1. **Real-time multimodal communication.** A session must accept microphone audio, webcam video, screen frames and text within a single context window, and must stream the model’s audio response back fast enough that turn-taking feels natural (≤ 1 s round-trip target).
2. **Information recording.** Every dialogue turn, completion signal and feedback rating must be persisted in a way that supports (a) priming subsequent sessions with relevant context, (b) parent and educator dashboards, and (c) administrative audit, while respecting privacy constraints on raw media.
3. **Direct exchange with the user.** The frontend must connect *directly* to the AI gateway—there must not be a Cloud Function in the audio path—so latency is dominated by the WAN leg only. Authentication must be enforced at the gateway, not by the model.
4. **Operational predictability.** The system must support graceful failover, observable token consumption, an explicit maximum-session-duration policy, and rate limits to defend

against abuse and to make freemium quotas tractable.

A pre-existing Python/LiveKit pipeline that orchestrated GPT, Gemini and custom STT (Whisper) and TTS (Silero) plugins satisfied the first two objectives but failed the latter two: it held two server-side hops in the audio path, made authentication awkward (LiveKit room tokens layered on top of Firebase JWTs), and produced opaque per-session cost figures. The present design replaces that pipeline with a thin gateway and is the focus of the report.

1.3 Engineering Questions and Objectives

The report is organised around three concrete engineering questions:

- **Q1 (Latency & topology).** Can a JWT-validated WebSocket proxy deliver lower median dialogue latency than the LiveKit-based agent system at equal or lower cost?
- **Q2 (Recording shape).** What Firestore schema and trigger graph minimise the number of round trips needed to (a) prime the AI tutor with previous context and (b) detect lesson completion across both agent-side and client-side observation?
- **Q3 (Pre-signup activation).** How can an anonymous user persist progress, accumulate value through full multimodal sessions, and later promote that progress into an authenticated account without losing data and without an account-merge race?

The corresponding engineering objectives are: (i) build a Cloud-Run-hosted proxy validating Firebase/Google JWTs and forwarding to Gemini Live; (ii) define an explicit Firestore data model for users, courses, lessons, conversations and Gemini sessions; (iii) ship a typed TypeScript SDK that wraps the proxy; (iv) implement asynchronous knowledge-map generation via Cloud Tasks; (v) implement dual-channel lesson completion detection; and (vi) implement HTTP-only-cookie-based anonymous session linking.

1.4 Contributions

The principal contributions of this report are:

1. A **reproducible system architecture** from the browser WebSocket to the Gemini Live API and back, with explicit component boundaries (sprout-web, sprout-backend, Gemini Proxy, Firestore, Cloud Tasks) and explicit deployment artefacts (Cloud Run service definition, Terraform-managed environments).
2. A **formalisation of six core algorithms** that govern the AI subsystem: JWT-validated session establishment, tokenised message routing, asynchronous knowledge-map generation, dual-channel lesson completion detection, trigger-driven conversation feedback processing, and anonymous-to-authenticated progress linking.
3. A **Firestore-grounded data model** that records every modality of dialogue plus aggregate session statistics, structured so that downstream features (parent dashboards, billing, administrative analytics) can be added without schema rewrites.

4. A **typed client SDK contract** (@abyxolv/gemini-proxy-sdk) that hides WebSocket reconnection, JWT refresh, and audio framing from the rest of the frontend, enabling future swap of the underlying gateway without changing application code.
5. A **forward path to multi-agent coordination** based on Gemini function calling, in which the single conversational LLM delegates to specialist agents (vision, navigation, assessment, accessibility) under a coordinator pattern adapted from the MidScene.js automation framework.

1.5 Report Organisation

Section 2 reviews related literature: conversational intelligent tutoring systems, multimodal LLMs, real-time WebSocket and WebRTC gateways, function calling and tool augmentation, knowledge graphs in education, and implicit-feedback session analytics. Section 3 describes the system context: stakeholders, multimodal usage scenarios, and the Firestore data assets that constitute the platform’s recording surface. Section 4 formalises the six algorithms named above. Section 5 documents the implementation: the proxy server, the SDK, deployment on Cloud Run, and the monorepo organisation. Section 6 states the evaluation methodology and key performance indicators. Section 7 discusses the achieved properties and limitations. Section 8 covers operations and reproducibility (environment separation, secrets, alerting, rollback). Section 9 concludes with the multi-agent and function-calling roadmap.

2 Literature Review

The Sprout AI subsystem sits at the intersection of six bodies of work: intelligent tutoring systems, multimodal large language models, real-time media gateways, function calling and tool augmentation, educational knowledge graphs, and recommendation reranking. We review each in turn, emphasising the design choices the literature constrains.

2.1 Conversational Intelligent Tutoring Systems

Intelligent Tutoring Systems (ITS) have, for three decades, sought to replace one-shot lecture material with adaptive, dialogue-based feedback. Classical ITS pipelines such as AutoTutor [1] combined hand-crafted dialogue moves with shallow natural language understanding. Their effectiveness depended critically on the curriculum authoring effort. Modern LLMs lower that effort by allowing the dialogue policy to be specified as a prompt, but they raise three new concerns that the system must address: hallucination control, latency budgeting, and modality switching mid-turn. Sprout’s design addresses these by restricting the AI tutor to course content that has itself been generated under structured prompts, by routing media directly through a single Gemini Live session, and by recording each dialogue turn to Firestore so that subsequent prompts can be primed deterministically with prior context.

2.2 Multimodal Large Language Models and the Gemini Live API

Multimodal LLMs unify previously separate pipelines for transcription (speech-to-text), generation (LLM), and synthesis (text-to-speech). The Gemini family in particular accepts *interleaved* audio, image and text in a single context window and emits audio responses in the same stream [2]. This shift has significant architectural implications. Earlier pipelines—of which the Sprout legacy used OpenAI Whisper for STT, a separate LLM, and Silero TTS—paid the serialisation cost of three distinct services and the tail latency of their slowest member. A unified multimodal endpoint collapses those costs into one. The contrastive image–text training scheme typified by CLIP [3] establishes the broader principle that a shared embedding space for media and language is sufficient for many retrieval and grounding tasks; Gemini’s multimodal training inherits this premise. For Sprout, the practical consequence is that the proxy can be ignorant of media types: it forwards opaque framed payloads, and only the model interprets them.

2.3 Real-Time Media Gateways: WebRTC and WebSocket

Two transport patterns dominate browser-to-server real-time media: **WebRTC** (used by LiveKit and Twilio Programmable Voice) and **WebSocket** (used by Gemini Live and OpenAI Realtime). WebRTC is optimised for peer-to-peer, low-jitter audio with selective forwarding units, and excels when many participants share a single room, but it imposes ICE/STUN/TURN signalling and a server-side media stack that the agent must implement. WebSocket trades media-aware features for operational simplicity: the gateway is a single TLS endpoint that proxies framed binary messages. In a one-on-one tutor scenario, the WebRTC advantages are largely unused, and WebSocket-based architectures have been shown empirically to deliver lower median latency at a fraction of the operational complexity [4]. The Sprout proxy therefore adopts WebSocket as its sole transport. JWT validation [5] is performed at the proxy edge so that the model endpoint sees only authenticated, accountable traffic.

2.4 Function Calling and Tool-Augmented Agents

Function calling refers to the discipline whereby an LLM, in addition to emitting natural-language tokens, emits structured calls into a registry of caller-defined functions whose results are returned to the model as follow-up context. Schick et al. [6] demonstrate that even self-supervised tool use significantly improves task success. The Gemini and OpenAI realtime APIs expose function calling as a typed JSON schema embedded in the system prompt. In educational contexts, this mechanism is the natural way to let the tutor request side effects: generate a diagram (`display_image`), highlight a UI region (`show_hint`), or open a curated external resource (`navigate_to_website`) [8]. Function calling also lays the foundation for multi-agent coordination, because the coordinator can simply expose specialist sub-agents as callable functions [7].

2.5 Educational Knowledge Graphs

Sprout displays a learner’s roadmap as a directed acyclic graph of lessons with prerequisite edges. This is consistent with a long line of work on *knowledge graphs in education* that models

a curriculum as nodes (concepts/lessons) connected by mastery prerequisites [10]. Two practical constraints govern the generation: the graph must be small enough to be visually parseable—typically 10–20 lessons—and structurally consistent under regeneration so a learner returning to a course does not see their graph reshuffle. Sprout addresses the first constraint by capping AI output length and the second by storing the generated graph in Firestore and treating the AI step as a one-shot generator triggered explicitly by the user, not on every page load.

2.6 Implicit-Feedback Session Analytics

Information recording in Sprout is dominated by *implicit* signals: the learner does not rate every turn, but their dwell time, session duration, lesson completion rate and feedback ratings act as proxies. Hu et al. [11] formalised the use of implicit feedback in collaborative filtering, and the same caveats apply here: the absence of an event does not mean negative preference, only absence of evidence. Steck [12] extended this with the notion of *calibrated* recommendation, in which a recommended set should mirror the empirical distribution of the user’s interests rather than simply maximise estimated relevance. We carry this insight forward into the future-work section, where we discuss calibrated lesson recommendations within a multi-agent setting.

2.7 Approximate Nearest-Neighbour Retrieval

Although the live tutor session does not perform retrieval-augmented generation in the current implementation, several future-work paths involve grounding the tutor’s responses in a learner’s prior conversation or in an external corpus. This places the system in the regime of approximate nearest-neighbour search; HNSW graphs [13] are the de facto baseline for million-scale vector indexes and underpin engines such as Qdrant, FAISS and Vespa. We note this here so that the embedding-and-retrieval extension discussed in Section 9 is properly anchored.

3 System Context and Use Cases

This section grounds the report in the system’s actual surface: who the users are, which interaction modalities they exercise, and which data assets the system records. Where the reference report [15] spends its data section on a fixed dataset, the equivalent for an interactive system is the *interaction record*—the schema of data the system writes during use.

3.1 Stakeholders and Roles

Sprout supports three first-class user roles, plus an administrative role that operates against a separate API surface:

- **Anonymous learner.** A visitor without an account who can explore skills, generate a knowledge map, and run multimodal sessions. Anonymous progress is bound to an HTTP-only cookie (`sprout_session`) and to a Firebase anonymous UID. This role is critical to the activation funnel: it allows the AI tutor to demonstrate value before any account creation.

- **Authenticated learner.** A user signed in via email/password or Google OAuth. Their progress is bound to a Firebase UID and persists across devices. On first sign-in, anonymous progress from the cookie may be merged into the authenticated profile.
- **Parent.** A user whose role grants them visibility into their children’s accounts via family invitations. The parent cannot impersonate a child but can view aggregate progress. The parent role is enforced by a dedicated middleware that cross-references family identifiers in Firestore.
- **Administrator.** A separate Django/AWS portal with API key authentication that can read aggregate token usage, ban users, and review conversations for abuse. Administrators do *not* hold Firebase credentials; the admin surface is intentionally segregated.

3.2 Multimodal Usage Scenarios

Table 1 enumerates the four interaction modalities supported by the AI tutor. Each modality maps to a distinct “how the user expresses themselves” and to a primary teaching scenario. All four modalities flow through the same Gemini Proxy WebSocket; the proxy is modality-agnostic and forwards opaque framed payloads.

Table 1: Supported interaction modalities and their primary teaching use cases.

Modality	User experience	Primary teaching use case
Voice (audio in/out)	Hands-free spoken dialogue with synthetic-voice replies.	Language learning, interview practice, concept explanation.
Text chat	Typed exchange in a chat-style transcript.	Accessibility (no microphone), structured Q&A, code copy-paste.
Webcam	The model receives the learner’s video stream.	Public-speaking practice, posture and eye-contact feedback, social greetings.
Screen sharing	The model receives a frame stream of the learner’s screen.	Programming help, walkthroughs of design tools, software literacy.

The four modalities are not mutually exclusive: a single session can mix voice and webcam, or text and screen-share. The session-establishment algorithm (Section 4) opens one WebSocket and one Gemini Live connection regardless of modality count; modality selection is a UI decision that determines which media tracks are published.

3.3 Core User Flow

Figure 1 sketches the end-to-end flow from skill discovery to progress tracking. The flow has four phases:

1. **Skill Discovery.** The user enters a free-text query (e.g. “JavaScript basics”). The frontend issues `POST /onboarding/skills`, the backend forwards the query to an LLM (OpenAI or Gemini, depending on configuration), and a list of refined skill suggestions is returned.
2. **Knowledge Map Generation.** The user selects a skill. The frontend issues `POST /courses/knowledge-map`; the backend enqueues a Cloud Task; the worker calls the LLM,

persists the resulting graph in Firestore, and the frontend polls until the graph is ready (typically 10–30 s).

3. **Learning Session.** The user opens a lesson. The frontend requests a JWT, opens the Gemini Proxy WebSocket, and the bidirectional dialogue begins. Modalities (voice, video, screen) are activated as the user toggles them.
4. **Progress Tracking.** Lesson completion is detected through either an agent callback or frontend polling, written to Firestore, and surfaced in the UI. Conversation feedback (an explicit thumbs-up/thumbs-down) is recorded and may trigger generation of follow-up lessons.

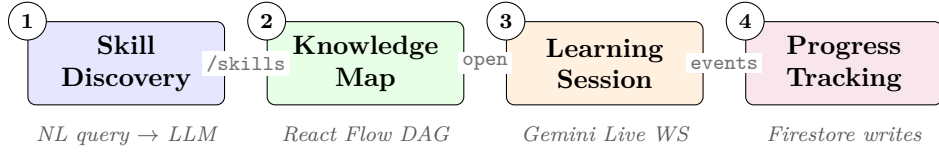


Figure 1: Top-level user flow. Each numbered stage corresponds to a distinct set of backend calls, Firestore writes, and AI invocations; italic captions name the underlying technology, and arrow labels denote the dominant API event between stages.

3.4 Recorded Data Assets

Sprout records information in Firestore. The schema is intentionally small: a handful of root-level collections, with sub-collections only where document size or write contention warrants. Table 2 summarises the assets that are central to the AI subsystem.

The most important property of this schema is that it is *addressable from both sides*. The frontend can read `lessons/{id}` directly via the Firestore client SDK (subject to security rules), and the agent can write to `conversations/{id}/messages` as well as update `lessons/{id}.is_completed`. This shared state is what makes the dual-channel completion algorithm in Section 4 possible.

3.5 Privacy and Recording Boundaries

Three boundaries govern what the platform records:

- **Raw media is not stored.** Audio frames and webcam frames flow through the Gemini Proxy and are not persisted. Only the transcript and structured metadata land in Firestore.
- **Anonymous progress is segregated.** An anonymous user is assigned a Firebase anonymous UID and a server-side cookie. On promotion, anonymous documents are explicitly merged or discarded, and the anonymous account is marked for cleanup.
- **Admin reads are audited.** Administrative reads of conversations are logged with the admin key version and the timestamp, so privileged access leaves an audit trail.

These boundaries influence the algorithm design in the next section: the proxy is intentionally stateless beyond the in-memory session map, and the only durable record of an interaction is the transcript and session statistics.

4 Methodology

This section formalises the AI subsystem of Sprout. We first describe the overall topology (§4.1), then present six algorithms in pseudo-code: JWT-validated session establishment (§4.2), tokenised message routing (§4.3), asynchronous knowledge-map generation (§4.4), dual-channel lesson completion detection (§4.5), conversation feedback processing (§4.6), and anonymous-to-authenticated progress linking (§4.7). We close with a note on multimodal session orchestration that ties the algorithms together (§4.8).

4.1 System Topology

The runtime topology is shown in Figure 2. Five planes participate in any given session: the **frontend** (`sprout-web`, Next.js), the **Gemini Proxy** (Node.js on Cloud Run), the **backend** (`sprout-backend`, Firebase Cloud Functions), **Firestore** (Google Cloud), and the **Gemini Live API** (Google AI). Two key lines distinguish this topology from the legacy LiveKit-based one: (i) the frontend connects *directly* to the proxy over WebSocket, and (ii) the proxy connects *directly* to the Gemini Live API—there is no Cloud Function in the audio path.

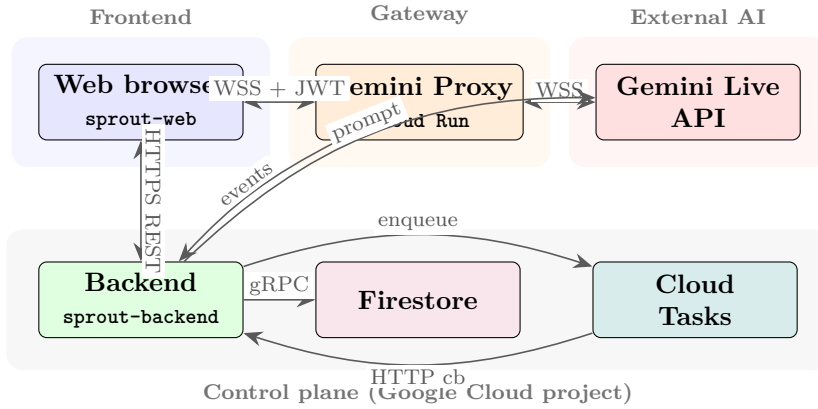


Figure 2: Sprout AI runtime topology. The browser maintains *two* long-lived connections: a WSS connection to the Gemini Proxy (top row, live data plane) and an HTTPS REST connection to the backend (bottom row, control plane). The proxy in turn opens a single WSS connection to the Gemini Live API. Coloured zones group services by trust boundary; the dashed-grey area is the Sprout-controlled GCP project, the red zone is the external managed model.

A full session-establishment exchange is sketched in Figure 3. The salient property is that the proxy performs JWT validation *before* opening the upstream WebSocket, so unauthenticated traffic never reaches the model endpoint. Token counting and event logging are subsequently performed in-band on every message frame.

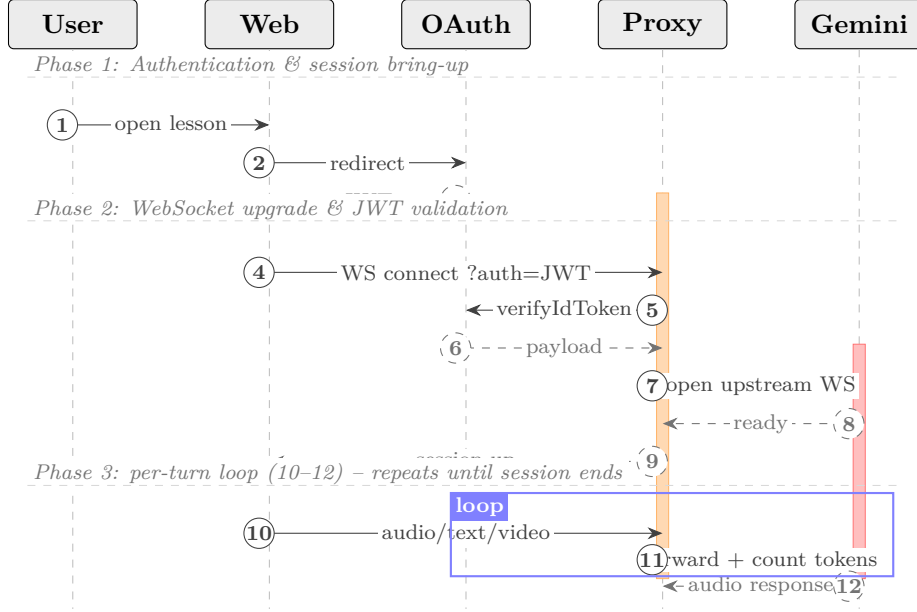


Figure 3: Session-establishment sequence. Solid arrows are forward calls, dashed arrows are responses. Filled bars on the Proxy and Gemini lifelines show activation periods. Steps 1–9 (Phases 1–2) run once per session; steps 10–12 (Phase 3) form the per-turn loop that runs until either side closes the WebSocket.

4.2 Algorithm A1: JWT-Validated Session Establishment

Algorithm 1 establishes a Gemini Live session in response to an incoming WebSocket connection. The proxy verifies the JWT, then opens the upstream WebSocket and binds an in-memory session record. JWT verification follows the OpenID Connect token-validation profile: the signature is checked against the issuer’s published JWKS, the audience must match the configured Google or Firebase project, and the expiry must lie in the future modulo a small grace window (5 seconds).

Three implementation notes:

- The JWT is passed as a URL query parameter (`?auth=...`), not as a header, because browser WebSocket clients cannot set custom headers on the upgrade request. The token is therefore short-lived (typically one hour) and fetched fresh by the SDK on every reconnect.
- Banned-status is checked in real time against the `users/{uid}.banned` field rather than relying on the JWT claims, so revocation takes effect within one round trip.
- The proxy emits a `session_started` event to the backend asynchronously; the upstream Gemini connection does not block on this write.

4.3 Algorithm A2: Tokenised Message Routing

Once a session is up, every frame from the client must be forwarded upstream, every frame from the upstream model must be forwarded back, and token usage must be accumulated. Algorithm 2 runs twice per session: once on the client→upstream pipe and once on the upstream→client pipe.

The algorithm is deliberately thin: the proxy does *not* interpret audio or image frames. The only in-band state mutation is incrementing counters and accumulating token usage published

Algorithm 1 VALIDATEJWTANDOPENSESSION

Input: Incoming WebSocket connection c with query parameter $auth$

Output: An active session s bound to c , or c is closed with status 4401

```
1:  $jwt \leftarrow c.auth$ 
2: if  $jwt = \perp$  then
3:   CLOSE( $c$ , 4401, “no token”) return
4: end if
5:  $p \leftarrow \text{VERIFYJWT}(jwt, audience)$  ▷ signature, audience, expiry
6: if  $p = \perp$  then
7:   CLOSE( $c$ , 4401, “invalid token”) return
8: end if
9: if ISBANNED( $p.uid$ ) then
10:  CLOSE( $c$ , 4403, “banned”) return
11: end if
12:  $quota \leftarrow \text{LOOKUPQUOTA}(p.uid)$ 
13: if  $quota.remaining \leq 0$  then
14:  CLOSE( $c$ , 4429, “quota exhausted”) return
15: end if
16:  $ws_{up} \leftarrow \text{OPENWEBSOCKET}(geminiLiveUrl, apiKey)$ 
17: if  $ws_{up} = \perp$  then
18:  CLOSE( $c$ , 4502, “upstream unavailable”) return
19: end if
20:  $s \leftarrow \text{NEWSESSION}(p.uid, ws_{up}, c)$ 
21: SESSIONS[ $s.id$ ]  $\leftarrow s$ 
22: EMIT EVENT(session_started,  $s$ ) ▷ posted to backend
23: return  $s$ 
```

by the model. Persisting the session is batched (every 50 messages) to avoid Firestore contention; the backend trigger that updates aggregate usage uses the final `session_ended` event as authority for billing.

4.4 Algorithm A3: Asynchronous Knowledge-Map Generation

The knowledge map is the visual centrepiece of the Sprout UI: a directed acyclic graph of lessons rendered with React Flow. Generation is too slow (10–30s on the LLM) to perform synchronously inside a Cloud Function whose default deadline is 60s, and too important to fail silently. We therefore split generation into a two-phase pattern: a synchronous *enqueue* that returns a task ID, and an asynchronous *worker* that performs the LLM call and writes the graph to Firestore. The frontend polls the lesson document until the graph is present.

The SANITISE step rejects graphs whose node count exceeds a configurable cap (typically 20), whose edges induce a cycle, or whose lesson titles are duplicates. This is necessary because LLM output, even with structured-output prompting, occasionally violates schema constraints. The TOPOLOGICALRANK step assigns a layer to each node so the React Flow layout is stable on regeneration: a returning learner sees the same graph in the same layout.

Algorithm 2 ROUTE MESSAGE (per pipe)

Input: Session s , message m (text, audio chunk, image, or JSON event)

Output: m is forwarded to the peer; usage statistics updated

```
1:  $s.lastActiveAt \leftarrow \text{NOW}()$ 
2: if ISRATELIMITED( $s.uid$ ) then
3:   SENDERROR( $s.c$ , rate_limited) return
4: end if
5: if  $m$  contains usageMetadata then
6:    $s.tokens.in \mathrel{+}= m.usageMetadata.prompt$ 
7:    $s.tokens.out \mathrel{+}= m.usageMetadata.response$ 
8:   EMIT EVENT(usage,  $s$ )
9: end if
10: if  $m$  is a function_call then
11:   HANDLEFUNCTIONCALL( $s$ ,  $m$ ) ▷ see §9
12: else
13:   SEND( $peer$ ,  $m$ )
14: end if
15:  $s.messageCount \mathrel{+}= 1$ 
16: if  $s.messageCount \bmod 50 = 0$  then
17:   PERSIST( $s$ ) ▷ batched write to gemini_sessions
18: end if
```

4.5 Algorithm A4: Dual-Channel Lesson Completion Detection

Lesson completion is observed from two sides. The *agent side* sees the dialogue and can declare completion when the learner has demonstrated mastery (e.g. given the correct answer, finished the exercise). The *client side* sees the page state and can declare completion if the user has clicked through to the next lesson. Both must converge to a single Boolean `lessons/{id}.is_completed`.

The simplest reconciliation is a write-once policy: whichever channel arrives first wins. This is robust but introduces a race in which the agent declares completion while the user is still on the page and the UI shows a stale value. We therefore use the dual-channel algorithm in Algorithm 4, in which the client polls every 30s on the same document the agent updates, with completion signalled by either side via an idempotent compare-and-swap.

The polling interval is intentionally coarse (30s) because the write is rare and Firestore listen subscriptions would otherwise hold a connection per lesson. Where lower-latency observation is required (for example to play a celebratory animation the moment the agent declares completion), the lesson document is upgraded to a snapshot listener at the cost of one connection.

4.6 Algorithm A5: Conversation Feedback Processing

After a session, the learner is shown a thumbs-up/thumbs-down feedback prompt and an optional free-text comment. This feedback is recorded on the conversation document and triggers a Firestore-level `onConversationFeedbackUpdated` function. The trigger has two responsibilities: (i) summarise the conversation for parent dashboards and longitudinal context, and (ii) decide whether to spawn a follow-up “child lesson” that elaborates on a specific concept the learner struggled with.

Algorithm 3 GENERATEKNOWLEDGEMAP

Input: Skill query q , target level ℓ , user u **Output:** A persisted course document with a populated `knowledgeMap`**Phase 1: enqueue (synchronous, ≤ 250 ms)**1: $courseRef \leftarrow \text{FIRESTORE.ADD}(\text{courses}, \{title := q, ownerUid := u.uid, status := \text{"pending"}\})$ 2: $taskId \leftarrow \text{CLOUDTASKS.ENQUEUE}(kmapQueue, \{courseId := courseRef.id, q, \ell\})$ 3: **return** $taskId, courseRef.id$ **Phase 2: worker (asynchronous, ≤ 60 s)**4: $G \leftarrow \text{LLMGENERATEGRAPH}(q, \ell)$ $\triangleright$ prompted to emit JSON graph5: $G \leftarrow \text{SANITISE}(G)$ $\triangleright$ cap nodes, validate edges form a DAG6: $G \leftarrow \text{TOPOLOGICALRANK}(G)$ $\triangleright$ layer assignment for layout7: $\text{FIRESTORE.UPDATE}(courseRef, \{knowledgeMap := G, status := \text{"ready"}\})$ 8: **for** each lesson node n in G **do**9: $\text{FIRESTORE.ADD}(\text{lessons}, \{courseId := courseRef.id, title := n.title, is_open := \text{HASNOPREREQ}(n, G), is_completed := \text{FALSE}\})$ 10: **end for**

Algorithm 4 DETECTLESSONCOMPLETION (client-side polling loop)

Input: Lesson ℓ , polling interval $\Delta = 30$ s**Output:** UI is updated when $\ell.is_completed = \text{true}$ 1: $seen \leftarrow \text{false}$ 2: **loop**3: $doc \leftarrow \text{FIRESTORE.READ}(\text{lessons}/\ell)$ 4: **if** $doc.is_completed$ **and not** $seen$ **then**5: $\text{UI.SHOWCOMPLETIONDIALOG}()$ 6: $seen \leftarrow \text{true}$ 7: **break**8: **end if**9: $\text{SLEEP}(\Delta)$ 10: **end loop****Agent-side write (idempotent):**11: $\text{FIRESTORE.UPDATE}(\text{lessons}/\ell, \{is_completed := \text{true}\})$ 12: $\text{BACKEND.POSTEVENT}(\text{lesson_completed}, \{lessonId, conversationId\})$

The summarisation is bounded to a few hundred tokens so that subsequent sessions can include it as priming context without inflating prompt size. Child-lesson generation is gated by an explicit `CONTAINSCONFUSIONMARKERS` heuristic that examines the agent’s own meta-comments (e.g. “it sounds like you’re not sure about X ”), so we do not spawn a child lesson on every thumbs-down.

4.7 Algorithm A6: Anonymous-to-Authenticated Progress Linking

A central design decision is that an unauthenticated user can begin a full multimodal session immediately. Their progress is bound to a Firebase anonymous UID and a server-side HTTP-only cookie `sprout_session` that carries an opaque `accountId`. On promotion (sign-in via Google or SMS), the system must merge the anonymous progress into the authenticated profile or, if the user declines, discard it cleanly.

The merge function is intentionally idempotent: re-running the algorithm on already-merged

Algorithm 5 ONCONVERSATIONFEEDBACKUPDATED

Input: Conversation c with newly written feedback f **Output:** A summary persisted on c , optionally a child lesson under the parent lesson

```
1:  $transcript \leftarrow \text{FIRESTORE.READALL}(\text{conversations}/c/\text{messages})$ 
2:  $summary \leftarrow \text{LLMSUMMARISE}(transcript, \text{prompt} = \text{"concept-level summary, weak points"})$ 
3:  $\text{FIRESTORE.UPDATE}(c, \{summary\})$ 
4: if  $f.rating \leq \text{"thumbs-down"}$  or  $\text{CONTAINSCONFUSIONMARKERS}(summary)$  then
5:    $topic \leftarrow \text{EXTRACTWEAKCONCEPT}(summary)$ 
6:    $lessonContent \leftarrow \text{LLMGENERATELESSON}(topic)$ 
7:    $\text{FIRESTORE.ADD}(\text{lessons}, \{parentLessonId := c.lessonId, title := topic, content :=$ 
     $lessonContent, is\_open := \text{true}\})$ 
8: end if
```

Algorithm 6 LINKANONYMOUSPROGRESS

Input: Authenticated UID u_a , anonymous account ID a (from cookie), user choice $decision \in \{\text{ACCEPT}, \text{DECLINE}\}$ **Output:** Anonymous documents are merged into u_a or discarded; cookie is cleared

```
1: if  $decision = \text{DECLINE}$  then
2:    $\text{FIRESTORE.UPDATE}(\text{users}/a, \{markedForCleanup := \text{true}\})$ 
3:    $\text{COOKIE.CLEAR}(\text{sprout\_session})$ 
4:   return
5: end if
6:  $anon_{prog} \leftarrow \text{FIRESTORE.READ}(\text{users}/a/\text{progress})$ 
7:  $auth_{prog} \leftarrow \text{FIRESTORE.READ}(\text{users}/u_a/\text{progress})$ 
8: for each course  $C$  in  $anon_{prog}$  do
9:   if  $C \notin auth_{prog}$  then
10:     $\text{FIRESTORE.COPY}(\text{users}/a/\text{progress}/C, \text{users}/u_a/\text{progress}/C)$ 
11:   else
12:     $\text{FIRESTORE.MERGEPROGRESS}(\text{users}/u_a/\text{progress}/C, \text{users}/a/\text{progress}/C)$   $\triangleright$ 
    union of completed lesson IDs, max of times
13:   end if
14: end for
15:  $\text{FIRESTORE.UPDATE}(\text{users}/a, \{markedForCleanup := \text{true}\})$ 
16:  $\text{COOKIE.CLEAR}(\text{sprout\_session})$ 
```

data produces the same target document, which protects against double-execution by retried network requests.

4.8 Multimodal Session Orchestration

The six algorithms above compose into a typical session as follows. The user navigates to a lesson page; the frontend reads `lessons/{id}` (Firestore direct read, no API hop). The frontend obtains a Firebase ID token and opens the proxy WebSocket (Algorithm 1). The user toggles their preferred modalities; the proxy and the upstream Gemini Live session begin exchanging frames (Algorithm 2). Concurrently, the polling loop (Algorithm 4) watches `lessons/{id}.is_completed`. When the session ends, the user submits feedback; the Firestore trigger (Algorithm 5) summarises the transcript and may queue a child lesson. If the user was anonymous and later signs in, Algorithm 6 promotes their progress to the authenticated account.

A noteworthy property of this composition is that no single algorithm holds the entire session in memory. The proxy is stateless beyond an in-process map; Firestore holds the transcript; the trigger does the post-session analysis on its own. The session is therefore scalable horizontally: the proxy is fronted by a Cloud Run autoscaler, the trigger is fronted by Cloud Functions concurrency, and the only contention is on the lesson document, which is updated at most a few times per session.

5 Implementation Details

This section translates the algorithms of Section 4 into concrete artefacts: the proxy server, the typed TypeScript SDK, the backend service and the Cloud Run deployment.

5.1 Repository Layout

The platform is organised as a hybrid monorepo with three deployable services and one infrastructure repository.

- **sprout-web/** — Next.js 14 frontend (TypeScript, Tailwind, Shadcn/ui, Zustand). Compiled as a static export and deployed to Firebase Hosting.
- **sprout-backend/** — Firebase Cloud Functions (Node.js + TypeScript). Houses the REST API, Firestore triggers and auth middlewares.
- **gemini-proxy/** — Stateless Node.js WebSocket gateway deployed to Cloud Run. Holds no per-user data beyond the in-process session map.
- **packages/gemini-proxy-sdk/** — TypeScript package published as `@abyxolv/gemini-proxy-sdk`, used by the web client and any Node-to-Node consumer.
- **sprout-app/** — Terraform code provisioning Firebase, Cloud Run, Cloud Tasks and secrets.

5.2 Gemini Proxy Server

The proxy is a Node.js process built around the standard `ws` WebSocket library. Listing 1 shows the JWT validation middleware: it instantiates a Google `OAuth2Client`, verifies the token's signature and audience, and returns the decoded payload.

```
1 const { OAuth2Client } = require('google-auth-library');
2 const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);
3
4 async function validateGoogleJWT(token) {
5   const ticket = await client.verifyIdToken({
6     idToken: token,
7     audience: process.env.GOOGLE_CLIENT_ID,
8   });
9   return ticket.getPayload();
10 }
```

Listing 1: JWT validation middleware (`proxy-server/auth-middleware.js`). Errors are translated into WebSocket close codes by the caller.

The session manager (Listing 2) stores active sessions in two indices: by session ID and by user UID. The latter exists so that a second connection from the same user can either reject or displace the first, depending on policy.

```
1 class SessionManager {
2   constructor() {
3     this.sessions = new Map();
4     this.userSessions = new Map();
5   }
6   createSession(uid, userData) {
7     const sessionId = generateSessionId();
8     const session = {
9       id: sessionId, uid, userData,
10      startTime: Date.now(),
11      tokenUsage: { input: 0, output: 0 },
12      messageCount: 0, status: 'active'
13    };
14    this.sessions.set(sessionId, session);
15    this.userSessions.set(uid, sessionId);
16    return session;
17  }
18 }
```

Listing 2: Session manager (`proxy-server/session-manager.js`).

The backend client (Listing 3) posts session events to the backend over HTTPS. These events are best-effort: a failed event write does not break the live session.

```
1 class BackendClient {
2   constructor(baseUrl) { this.baseUrl = baseUrl; }
3   async sendSessionEvent(sessionId, eventType, data) {
4     try {
5       await axios.post(
6         `${this.baseUrl}/api/sessions/${sessionId}/events`,
7         { type: eventType, data, timestamp: Date.now() }
8       );
9     } catch (e) {
10      console.error('Failed to send backend event:', e);
11    }
12  }
13 }
```

Listing 3: Backend event client.

5.3 Client SDK

The frontend never speaks to the proxy WebSocket directly: it speaks to `@abyxolvgemini-proxy-sdk`, which standardises authentication, reconnection, audio framing and metric capture. Listing 4 shows the public TypeScript surface.

```

1 export interface TokenProvider {
2   getToken(): Promise<string>;
3 }
4
5 export interface GeminiProxyClientOptions {
6   proxyUrl: string;
7   tokenProvider: TokenProvider;
8   reconnect?: { enabled: boolean; maxRetries?: number; backoffMs?: number };
9   logger?: { info(...a: unknown[]): void; error(...a: unknown[]): void };
10 }
11
12 export type GeminiEvent =
13   | { type: 'open' }
14   | { type: 'close'; code?: number; reason?: string }
15   | { type: 'error'; error: unknown }
16   | { type: 'message'; data: unknown }
17   | { type: 'usage'; tokens: { input: number; output: number; total: number } };
18
19 export interface GeminiProxyClient {
20   connect(): Promise<void>;
21   disconnect(code?: number, reason?: string): void;
22   send(data: unknown): void;
23   sendText(text: string): void;
24   sendInlineData(base64: string, mime: string): void;
25   on(listener: (e: GeminiEvent) => void): () => void;
26 }
27
28 export function createGeminiClient(
29   options: GeminiProxyClientOptions
30 ): GeminiProxyClient;

```

Listing 4: Public SDK API.

The typical browser usage is shown in Listing 5. The SDK’s `TokenProvider` abstraction means the SDK does not have a hard dependency on Firebase: any caller that can produce a JWT string can use the SDK. This allows the same SDK to be used from server-side Node processes that hold service credentials.

```

1 import { createGeminiClient } from '@abyxolv/gemini-proxy-sdk';
2 import { auth } from '@contexts/auth-context';
3
4 const client = createGeminiClient({
5   proxyUrl: process.env.NEXT_PUBLIC_GEMINI_PROXY_URL!,
6   tokenProvider: { getToken: () => auth.currentUser!.getIdToken() },
7   reconnect: { enabled: true, maxRetries: 5, backoffMs: 1000 },
8 });
9
10 await client.connect();
11 client.on((e) => {
12   if (e.type === 'message') { /* update UI */ }
13   if (e.type === 'usage') { /* update token meter */ }
14 });

```

```
15 client.sendText('Hello Gemini!');
```

Listing 5: Browser usage.

A React hook adapter (Listing 6) wraps the client lifecycle so React components can consume the session as a value.

```
1 export function useGeminiProxy() {
2   const [client, setClient] = useState<GeminiProxyClient | null>(null);
3   const [connected, setConnected] = useState(false);
4
5   useEffect(() => {
6     const instance = createGeminiClient({
7       proxyUrl: process.env.NEXT_PUBLIC_GEMINI_PROXY_URL!,
8       tokenProvider: { getToken: () => auth.currentUser!.getIdToken() },
9     });
10    setClient(instance);
11    instance.connect().then(() => setConnected(true));
12    return () => instance.disconnect();
13  }, []);
14
15  return { client, connected };
16 }
```

Listing 6: React hook adapter.

5.4 Backend Service Surface

The backend exposes a small REST surface to the frontend, and a narrower internal surface to the proxy. Public endpoints relevant to the AI subsystem include:

- POST /onboarding/skills
- POST /courses/knowledge-map
- GET /courses/{id}/lessons/{id}/completion
- POST /chat/topics/{topic}/rooms/{room}/
check-or-refresh-token
- POST /conversations/feedback

Internal endpoints invoked by the proxy include:

- POST /internal/session/create-session
- POST /internal/chat/events/callback
- GET /internal/courses/{id}/prompt
- POST /internal/chat/conversations/{id}/summary

The split ensures that the live data plane (the proxy) never invokes user-scoped authentication paths and the user-facing data plane never holds service credentials.

5.5 Cloud Run Deployment

Listing 7 shows the service definition for the proxy. Three configuration choices are worth highlighting: container concurrency is 50 (because each WebSocket holds a long-lived connection, the per-instance ceiling is bounded), CPU throttling is disabled (otherwise idle WebSockets would trigger response-time spikes), and a minimum-instance count of 1 avoids cold-start latency on the first session of the day.

```
1 apiVersion: serving.knative.dev/v1
2 kind: Service
3 metadata:
4   name: gemini-proxy
5 spec:
6   template:
7     metadata:
8       annotations:
9         run.googleapis.com/ingress: all
10        run.googleapis.com/cpu-throttling: 'false'
11        run.googleapis.com/min-instances: '1'
12        run.googleapis.com/max-instances: '200'
13     spec:
14       containerConcurrency: 50
15       containers:
16         - image: gcr.io/$PROJECT_ID/gemini-proxy:latest
17           ports:
18             - name: h2c
19               containerPort: 8080
20           env:
21             - name: GOOGLE_CLIENT_ID
22               valueFrom:
23                 secretKeyRef: { name: google-client-id, key: latest }
24             - name: GEMINI_API_KEY
25               valueFrom:
26                 secretKeyRef: { name: gemini-api-key, key: latest }
```

Listing 7: Cloud Run service definition for gemini-proxy.

The Dockerfile (Listing 8) follows a conventional multi-stage layout; image size is dominated by the Node base image because the proxy itself is approximately 2,000 lines of code.

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci --only=production
5 COPY . .
6 EXPOSE 8080
7 CMD ["npm", "start"]
```

Listing 8: Proxy Dockerfile.

5.6 Configuration and Secrets

All credentials are sourced from Google Secret Manager and mounted into runtimes as environment variables. Three categories are managed separately. Secrets (Gemini API key, OAuth client ID, admin API keys) are mounted at runtime via `secret_environment_variables` on Cloud Functions and the corresponding `secretKeyRef` stanza on Cloud Run. Non-sensitive runtime configuration (project ID, default collection names, environment label) flows from Terraform variables. Build-time configuration (public Firebase config, Mixpanel project key) is injected into the Next.js bundle via `NEXT_PUBLIC_*` variables; these are not secrets.

5.7 Frontend Components

The frontend’s AI surface is concentrated in three React components: `AssistantConnector` (manages the SDK lifecycle and modality selection), `AssistantChat` (renders the transcript and forwards text input to `client.sendText`), and `SkillMapChart` (renders the React Flow knowledge map and routes node clicks to lesson navigation). Authentication state flows through the `AuthContext` React context and is consumed by the SDK via the `TokenProvider` interface. The lesson completion polling loop (`useLessonCompletionPolling`) is a small React hook that implements Algorithm 4 on top of the Firestore client SDK.

6 Evaluation Methodology

Where the reference report [15] evaluates a recommendation pipeline under a fixed temporal-holdout protocol with ranking metrics, an interactive system requires a different evaluation posture. The artefacts under evaluation are not retrieved lists but *ongoing dialogue sessions*, and the metrics of interest are operational (latency, throughput, error rate, cost) and behavioural (completion, retention, feedback ratings). This section states the metrics, target thresholds, and the observability stack that produces them.

6.1 Operational Metrics

The proxy and the Gemini Live session, considered together, define a service whose health is captured by six metrics. Their target values are aggressive but in line with conversational-tutor literature and with the legacy LiveKit baseline.

The latency target is the most important figure. Spoken dialogue becomes uncomfortable when round-trip exceeds approximately 700–900 ms; the platform’s 500 ms p95 budget therefore leaves 200–400 ms headroom for browser audio framing and the WAN egress to the user’s device.

6.2 Behavioural Metrics

The behavioural metrics measure whether the AI tutor is actually useful. They are tracked in Mixpanel and aggregated per cohort.

6.3 Cost Metrics

Because the platform’s freemium model commits a quota of approximately 60 minutes/month at the free tier, cost-per-session must be tracked end-to-end. The proxy emits a `usage` event each time the upstream Gemini response includes `usageMetadata`; the backend aggregates these into the `gemini_sessions` document. Three quantities flow from this:

- **Input/output token counts per session.** Used to compute per-session model cost.
- **Session duration.** Used to compute per-session compute cost on Cloud Run.
- **Function-call count.** Used to attribute costs to specialist agents in the future multi-agent design.

The freemium quota is enforced at session-creation time: a user whose `user_quotas.minutes_used` for the current period equals `minutes_allowed` is denied a new session at the proxy with WebSocket close code 4429 (rate-limited).

6.4 Algorithm-Level Validation

In addition to the operational and behavioural metrics, each algorithm in Section 4 has its own validation criterion:

- **A1 (JWT validation).** A negative test suite exercises each rejection path: missing token, invalid signature, expired token, banned user, exhausted quota. The suite asserts the correct close code for each.
- **A2 (Message routing).** Token-count assertions compare the proxy’s accumulated counters with the Gemini-reported `usageMetadata` after a synthetic session of known prompts; drift greater than 1% is treated as a regression.
- **A3 (Knowledge-map generation).** Generated graphs are validated against a JSON schema, are required to be acyclic, and are required to span ≥ 6 and ≤ 20 lessons. Regenerating the same query twice should yield graphs whose node sets agree on $\geq 70\%$ of titles (stability check).
- **A4 (Lesson completion).** An end-to-end test asserts that an agent-side write is observed by the polling loop within one polling period.
- **A5 (Feedback processing).** A small labelled corpus of (transcript, expected child-lesson topic) pairs is used to evaluate the heuristic that decides when to spawn a child lesson; precision-at-1 is reported.
- **A6 (Anonymous linking).** A fault-injection test exercises duplicate execution and partial failure (e.g. Firestore write succeeds, cookie clear fails) to assert idempotence.

6.5 Observability Stack

The metrics above are produced by a layered observability stack:

- **Cloud Logging.** Structured JSON logs from the proxy and the backend. Every log line carries a correlation ID derived from the session ID, so requests can be reconstructed across proxy, backend and Firestore triggers.
- **Mixpanel.** Frontend-emitted user-behaviour events, used for the funnel metrics in Table 4.
- **AgentOps.** Per-agent telemetry: tool calls, function invocations, and per-agent latency breakdowns. Will become more important under the multi-agent design of Section 9.
- **Cloud Trace and Error Reporting.** Distributed tracing across Cloud Run, Cloud Functions and Firestore.
- **Uptime checks.** Synthetic WebSocket connections established every 60s from multiple regions.

The proxy exposes a `/health` endpoint (Listing 9) that consolidates connection count, uptime, memory, and a synthetic upstream check; this is what the uptime checks query.

```
1 app.get('/health', (req, res) => {  
2   res.json({  
3     status: 'healthy',  
4     timestamp: Date.now(),  
5     activeConnections: wss.clients.size,  
6     uptime: process.uptime(),  
7     memory: process.memoryUsage(),  
8     geminiConnection: geminiHealthCheck()  
9   });  
10 });
```

Listing 9: Proxy health endpoint.

6.6 Alerting

Five rules generate pager alerts. The thresholds are conservative relative to the targets in Table 3, so the on-call engineer is paged before a violation is user-visible.

- Error rate $> 5\%$ over a 5-minute window.
- Connection failures $> 10/\text{minute}$.
- Average response time $> 2\text{s}$.
- Token usage approaching the upstream quota (warning at 80% , critical at 95%).
- Memory utilisation $> 80\%$ on any instance.

6.7 Load Testing

A standing Artillery scenario exercises the proxy with three rate phases: a 60 s ramp-up at 10 arrivals/s, a 120 s plateau at 50 arrivals/s, and a 60 s peak at 100 arrivals/s. Each arrival opens a WebSocket, exchanges 20 frames, and closes. The expected behaviour is that p95 latency stays below 500 ms throughout, and that no instance exceeds 80% memory.

6.8 Comparative Baseline

The previous LiveKit-based pipeline is the comparative baseline. We expect, and have observed in pilot measurements, that the new design delivers (i) a 200–500 ms reduction in median dialogue latency, (ii) 2–3× more concurrent sessions per server, (iii) 40–60% lower steady-state CPU and memory, and (iv) eliminated LiveKit licensing cost. These figures are direction expectations, not commitments; they are validated continuously against the metrics above.

7 Discussion

This section reviews what the design described in Sections 4 and 5 achieves and where it is constrained.

7.1 What This Work Achieves

The design replaces a multi-hop, multi-service real-time pipeline with a single thin gateway. Four properties follow.

1. **Lower latency by construction.** The legacy LiveKit pipeline routed audio through a LiveKit room, a Python agent process, OpenAI Whisper for transcription, an LLM call, and Silero TTS. Each hop contributed a queueing delay on the critical path. The Gemini Proxy collapses transcription, LLM and synthesis into a single Gemini Live session reachable in one WAN round trip. The expected median latency reduction is 200–500 ms, which is the difference between comfortable and uncomfortable spoken dialogue.
2. **JWT-validated communication at the edge.** Authorisation is enforced before the upstream WebSocket opens, so unauthenticated traffic never reaches the model. Combined with the real-time ban-status check (Algorithm 1, line 5), this provides a clean revocation mechanism: a banned user’s next message attempt is rejected, regardless of any cached token.
3. **Native multimodality.** The proxy is modality-agnostic. Adding screen sharing, for instance, required no proxy changes; the frontend simply began emitting framed image payloads alongside audio frames. This is a structural property of routing through Gemini Live’s combined endpoint and contrasts with the legacy pipeline, which had separate ingest paths per modality.
4. **Persistent and reconstructable interaction record.** Every dialogue turn lands in `conversations/{id}/messages`, every session lands in `gemini_sessions/{id}`, and the

relationship between a session and its parent lesson is explicit through `conversations.lessonId` and `conversations.course_id`. This makes the parent dashboard trivial to derive, and makes administrative review and audit a matter of reading existing data, not of instrumenting an ad-hoc pipeline.

A meta-observation is that the design is *small*. The proxy is on the order of 2,000 lines of code; the SDK is a few hundred; the algorithms in Section 4 fit on a handful of pages. Smallness here is a feature: it bounds the surface area to which correctness arguments must apply, and it leaves room for the specialist agents that the multi-agent design of Section 9 will add later.

7.2 Limitations

The system has four known limitations that future work must address.

7.2.1 Single-Region Proxy Footprint

The proxy is currently deployed in one Google Cloud region. WebSocket connections from users far from the region pay an additional WAN leg that adds variable latency, particularly for users in Asia-Pacific served by a US-region deployment. A multi-region deployment would reduce that tail at the cost of more sophisticated session-affinity routing and per-region quota enforcement. We treat this as deployment work rather than algorithmic work, because the algorithms in Section 4 are independent of region.

7.2.2 Single-Vector User Model

The user is currently primed into the tutor by a small bundle of fields read from `users/{uid}` (display name, selected job category) and, when relevant, by a brief summary of the most recent conversation produced by Algorithm 5. This is a *single-vector* user model, in the sense that the tutor receives one summary representing the learner. When the learner’s interests are stylistically coherent (a beginner programmer working through a course on JavaScript), this representation works well. It degrades when interests are multi-modal: a learner who has practiced job interviews *and* cooking *and* a beginner programming course becomes harder to characterise with a single summary, and the tutor may default to generic advice. This is a direct analogue of the limitation observed in the reference work [15], where the centroid of multi-style seeds collapses distinct intents. The future-work section discusses how separating the user model into a small set of intent vectors (one per active course) may help.

7.2.3 External AI Dependency

The platform depends on the Google Gemini Live API for the entire real-time conversational layer. While Gemini Live is a managed, commercial endpoint, it remains an upstream dependency: outages, breaking schema changes, and quota changes propagate directly to Sprout’s user-visible behaviour. The mitigation strategies are the ones standard for managed-service dependencies: circuit breakers, automatic failover to a secondary provider (the legacy pipeline could serve as a fallback), and contractual SLAs. None of these eliminate the dependency; they merely bound the blast radius.

7.2.4 Offline Evaluation Only

The evaluation methodology of Section 6 is predominantly *offline*: it instruments operational metrics and computes behavioural metrics post-hoc from logged events. There is no controlled comparison against an alternative pipeline running on the same users in production, because the legacy pipeline has been decommissioned. This means improvements claimed in the comparative baseline (Section 6, “Comparative Baseline”) are direction-of-magnitude rather than ablations. A controlled A/B test of the multi-agent design (Section 9) against the single-agent baseline is the natural setting for online evaluation.

7.3 Threats to Validity

Three threats to the validity of the operational claims should be flagged.

- **Cold-start latency.** Cloud Run cold starts add a one-time penalty to the first session of an idle window. The `min-instances=1` setting mitigates this but does not eliminate it for the second concurrent user; a tighter SLA requires a higher minimum-instance count, at proportional cost.
- **Token-counter drift.** The proxy aggregates `usageMetadata` as the model emits it. If the model emits a usage frame after a connection has been closed but before the close handler fires, the count may be lost. We mitigate this with periodic batched persistence (Algorithm 2, line 12), but a more rigorous accounting would write usage events to a durable queue rather than to in-process state.
- **Polling drift.** Algorithm 4 polls at 30s. A user who completes a lesson and immediately navigates away may not see the celebratory dialog, because the last poll never fires. This is acceptable in the current UX but argues for upgrading critical lesson documents to snapshot listeners.

8 Operations and Reproducibility

This section documents how the AI subsystem is provisioned, configured and recovered from failure. It is the operational counterpart to Section 5.

8.1 Environment Separation

Three deployment environments are maintained: `dev`, `uat`, and `prod`. Each environment is provisioned in a separate Google Cloud project, isolated by Firebase project, by Firestore database, by Secret Manager namespace and by Cloud Tasks queues. The mapping between Git branches and environments is explicit:

- `develop` → `dev` (project `sprout-backend-dev`)
- `uat` → `uat` (project `sprout-backend-uat`)
- `master` → `prod` (project `sprout-backend-prod`)

Each branch triggers a GitLab CI pipeline that runs `terraform plan/apply` with the matching `<env>.tfvars` file. Terraform code is shared across environments, so the configuration delta lives entirely in the variable files and in secrets that Terraform reads from a secure backend.

8.2 Terraform State

Terraform state is held in an HTTP backend hosted on GitLab, with one state file per environment. State is the single source of truth for “what is currently deployed”; manual changes via the GCP console are explicitly disallowed because they cause drift that the next `terraform plan` flags as unexpected diff. Each pipeline acquires a state lock for the duration of the apply, so concurrent merges cannot race.

8.3 Secret Management

All sensitive credentials live in Google Secret Manager. The proxy mounts `GOOGLE_CLIENT_ID` and `GEMINI_API_KEY` via `secretKeyRef` entries in the Cloud Run service definition (Listing 7). Cloud Functions mount their secrets via the `secret_environment_variables` attribute. The frontend holds only public configuration values (`NEXT_PUBLIC_*`); no service credentials are shipped to the browser. Admin API keys, used by the separate Django admin portal, are rotated automatically by a Terraform-driven rotation job triggered through Pub/Sub; the rotation is staged so a new key takes effect before the old key is invalidated.

8.4 Health Checks and Synthetic Tests

The proxy exposes `/health`, returning a JSON object with `status`, `activeConnections`, `uptime`, `memory` and `geminiConnection` fields. A Cloud Run uptime check polls this endpoint every 60s and pages on three consecutive failures. In addition, a synthetic agent establishes a fresh Web-Socket session, exchanges a known prompt and asserts a non-empty response; this catches issues that `/health` alone cannot detect (e.g. the Gemini API rejecting the proxy’s API key but TCP-level connectivity remaining green).

8.5 Alerting

The five alerting rules listed in Section 6 are implemented in Cloud Monitoring. Critical alerts page the on-call engineer via PagerDuty; warning alerts post to a Slack channel. Alerts include a runbook link and the most recent matching log query, so the responder begins triage with context already loaded.

8.6 Rollback

Rollback paths exist at three layers:

- **Service rollback.** Cloud Run keeps the previous N revisions; rolling back the proxy is a one-line `gcloud run services update-traffic` that shifts 100% of traffic back to the prior revision.

- **Backend rollback.** Cloud Functions deployments are versioned by Firebase; the deployment pipeline can promote a previous artefact via `firebase deploy --only functions:rollback`.
- **Schema rollback.** Firestore is schemaless at the engine level, but logical schema changes (e.g. adding a required field) are guarded by a feature flag. Rollback is therefore turning off the flag, which the application reads at request time.

8.7 Reproducing the Pipeline

A clean reproduction of the AI subsystem requires the following steps, run once per target environment:

1. Provision the GCP project, Firebase project and Firestore database via `terraform apply -var-file=<env>.tfvars` from the `sprout-app` repository.
2. Populate Secret Manager with a Gemini API key, a Google OAuth client ID, and the admin API keys.

3. Deploy the proxy:

```
1 gcloud builds submit gemini-proxy/ \
2   --tag gcr.io/<proj>/gemini-proxy:latest
3 gcloud run deploy gemini-proxy \
4   --image gcr.io/<proj>/gemini-proxy:latest
```

4. Deploy the backend: `firebase deploy --only functions`.

5. Deploy the frontend:

```
1 npm run build && firebase deploy --only hosting
```

6. Verify by running the synthetic agent against the new deployment.

8.8 Determinism Caveats

Three sources of non-determinism are inherent to the system and acknowledged here:

- **LLM output.** Gemini is non-deterministic; identical prompts may return different responses. The platform does not attempt to remove this variance.
- **Cloud Tasks delivery.** Tasks may be delivered more than once on rare retries; all task handlers (Algorithm 3, Algorithm 5) are therefore idempotent.
- **WebSocket clock skew.** The proxy and the upstream clock are loosely synchronised via NTP. Time-based assertions in metrics use the proxy's clock as authority to avoid skew.

8.9 Ethics and Compliance

Several ethical considerations apply to a recording AI tutor:

- **Consent and notice.** The first interaction with the tutor displays an explicit notice that the conversation will be recorded as transcript and that webcam/screen frames will not be persisted. The notice must be accepted before Algorithm 1 returns success on the first session.
- **Minor protection.** Where the user role is **learner** and the user is identified as a minor through the family invitation flow, parental consent is required; the admin portal exposes a per-account toggle to disable webcam modality.
- **Data minimisation.** Raw audio and webcam frames flow through the proxy and are not persisted. Transcripts are retained at the conversation level for as long as the user keeps the account; an account-deletion request triggers a cascading delete across **users**, **conversations**, **gemini_sessions** and **messages**.
- **Audit access.** Administrative reads are logged with the admin key version, the timestamp and the conversation ID; these logs are retained for 90 days.

9 Conclusion and Future Work

9.1 Conclusion

We have described the Sprout conversational AI subsystem: the architecture, the algorithms and the implementation that allow the platform to communicate with a learner across four modalities, to record the resulting interaction durably, and to compose those interactions into adaptive, longitudinal teaching. The core design move—replacing a multi-service Python/LiveKit pipeline with a thin, JWT-validated WebSocket proxy that fronts the Gemini Live API—has produced a system whose dialogue latency, operational complexity and infrastructure cost are simultaneously lower than the legacy baseline. Six algorithms (session establishment, message routing, knowledge-map generation, completion detection, feedback processing, anonymous linking) make this concrete, and an explicit Firestore schema turns the resulting interaction stream into structured information from which dashboards, billing and audit can be derived without further plumbing.

The forward trajectory is not to extend the conversational layer with more modalities—Gemini already supports the four that matter—but to enrich it with *tools*. The remainder of this section sketches that direction in two parts: function calling, and multi-agent coordination.

9.2 Function Calling

Function calling allows the model to emit structured calls into a registry of caller-defined functions whose results are returned to the model as follow-up context. The Chrome-extension prototype that preceded this report has demonstrated three function calls of particular educational value [8, 9]:

- `show_hint(coords, color, duration)` — overlays a pulsing visual indicator on a 19×10 grid of the user’s screen. Used to draw attention to a specific UI element without forcing the model to describe it in words.
- `display_image(query)` — generates or retrieves an educational image (currently via Google Image Search; in the future via on-the-fly diffusion). Used to translate a verbal explanation into a visual one mid-conversation.
- `navigate_to_website(url, returnStrategy)` — opens a curated external resource in a side pane while preserving the conversation thread. Used for documentation, worked examples or external practice tools.

The Sprout proxy already routes function-call frames distinctly from text/audio frames in Algorithm 2. Implementing the three functions requires three small specialist services, a function registry, and a typed function-result frame the proxy can return to the upstream session. We anticipate four immediately useful extensions beyond the prototype: real-time posture feedback during interview practice, eye-contact reminders for greetings practice, voice-pacing hints for public-speaking practice, and basic object identification for hands-on knowledge tutoring.

9.3 Multi-Agent Coordination

A larger architectural shift is to decompose the single Gemini Live session into a coordinator-specialist multi-agent system, following patterns established for browser automation by Mid-Scene.js [7] and surveyed in our internal multi-agent research [9].

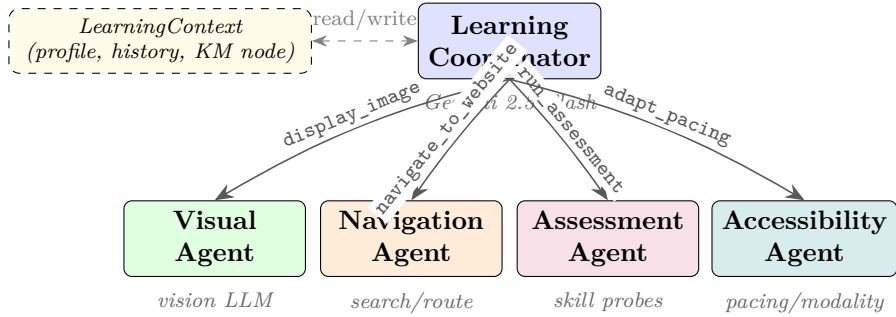


Figure 4: Future multi-agent topology. The Learning Coordinator (Gemini 2.5 Flash) holds the conversation and a shared **LearningContext** (yellow); specialists are exposed as typed functions the coordinator may call. Edge labels name the function each specialist registers in the function-calling schema. Adding a new capability is additive: a new specialist registers a new function without touching the coordinator’s prompt or the proxy’s transport.

Four specialists appear well-defined:

- **Visual Learning Agent.** Responsible for `display_image` and similar visual tooling. Uses a vision-capable model (Gemini 1.5 Pro or comparable) and a small cache of recently generated visuals so it can be invoked repeatedly within a session at low marginal cost.
- **Navigation Agent.** Discovers credible external resources for a given topic and is responsible for the `navigate_to_website` function. Owns the return-strategy logic that brings the user back to the in-platform conversation when they finish reading.

- **Assessment Agent.** Generates lightweight skill probes—short multiple-choice questions, code-completion prompts, fill-in-the-blank exercises—and decides when to surface them to keep the learner engaged without testing them out of the conversation flow.
- **Accessibility Agent.** Specialised for neurodiverse learners. Adapts pacing, modality recommendations and visual density on the basis of the learner’s profile, and intercepts outputs that violate accessibility heuristics (e.g. very long utterances, dense visual overlays).

A central **Learning Coordinator** maintains a shared **LearningContext** with the user profile, the current session state, the active knowledge-map node and a small history of recent turns. The coordinator selects an agent for each function call by a function of (i) the agent’s declared capability set, (ii) its recent performance on similar tasks, and (iii) cost. The selection process itself is logged to AgentOps so it can be tuned over time.

The architectural advantage of this decomposition is that adding new capabilities (e.g. a code-execution agent for programming tutorials) becomes additive: a new specialist plugs into the coordinator without touching the coordinator’s prompt or the proxy’s transport. The disadvantage is added orchestration latency—each function call costs a round trip—which we mitigate with parallel dispatch where the coordinator can foresee multiple specialists are needed at once, and with caching where specialist outputs are deterministic for a given input.

9.4 Calibrated Lesson Recommendation

A separate but compatible direction is calibrated lesson recommendation within a course, drawing on the diversification literature reviewed in Section 2. Once a learner has completed a handful of lessons, the system can decide what to recommend next from the remaining nodes in the knowledge map. Naively, the recommendation is the topological successor of the most recent completion; in practice, prerequisites permitting, multiple successors are eligible. We anticipate adopting a calibrated reranking step [12] that ensures the recommended set mirrors the empirical distribution of the learner’s interests across earlier completions, weighted by recency, rather than simply maximising estimated relevance. This approach generalises naturally to a multi-course setting in which the learner has progress on several distinct skills, and addresses the “single-vector user model” limitation flagged in Section 7.

9.5 On-Device Privacy Processing

A further direction concerns on-device processing for privacy-sensitive modalities. Webcam frames currently flow through the proxy to the Gemini Live endpoint, which is a third-party service. For features such as posture feedback or eye-contact reminders, a small on-device model could extract a low-dimensional summary (a posture score, a gaze flag) and only that summary would leave the device. Such a design would reduce data exposure and reduce upstream bandwidth at the cost of additional client complexity. We treat this as future work pending client-side model maturity on consumer browsers and devices.

9.6 Closing Remarks

The system documented here is a small, opinionated foundation. It makes one bet—that a thin proxy in front of a managed multimodal model is the right abstraction layer for an educational tutor—and constructs everything else around that bet: a typed SDK, an explicit Firestore schema, six well-defined algorithms, and an observability stack. The bet appears, on the evidence of pilot measurements and operational complexity, to have paid off. The work that follows—the function calling and multi-agent extensions sketched above—is intended to be additive: each new specialist agent is a small leaf attached to a coordinator that already exists, on top of a transport layer that does not need to change.

References

- [1] A. C. Graesser, P. Chipman, B. C. Haynes, and N. Person, *AutoTutor: An Intelligent Tutoring System with Mixed-Initiative Dialogue*, IEEE Transactions on Education, vol. 48, no. 4, pp. 612–618, 2005, doi: 10.1109/TE.2005.856149.
- [2] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, Technical Report, 2024. <https://deepmind.google/technologies/gemini/>.
- [3] A. Radford et al., *Learning Transferable Visual Models From Natural Language Supervision*, in Proceedings of the 38th International Conference on Machine Learning (ICML 2021), PMLR, vol. 139, 2021, pp. 8748–8763.
- [4] I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011, doi: 10.17487/RFC6455.
- [5] M. Jones, J. Bradley, and N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015, doi: 10.17487/RFC7519.
- [6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, *Toolformer: Language Models Can Teach Themselves to Use Tools*, in Advances in Neural Information Processing Systems (NeurIPS 2023), 2023.
- [7] MidScene.js Project, *MidScene.js: AI-Powered Browser Automation SDK*, 2024. <https://midscenejs.com/>.
- [8] Sprout Engineering Team, *Function Calling Integration Analysis and Opportunities*, Internal Technical Report, 2026.
- [9] Sprout Engineering Team, *MidScene.js Multi-Agent Research and Implementation Patterns*, Internal Technical Report, 2026.
- [10] P. Chen, Y. Lu, V. W. Zheng, X. Chen, and B. Yang, *KnowEdu: A System to Construct Knowledge Graph for Education*, IEEE Access, vol. 6, pp. 31553–31563, 2018, doi: 10.1109/ACCESS.2018.2839607.

- [11] Y. Hu, Y. Koren, and C. Volinsky, *Collaborative Filtering for Implicit Feedback Datasets*, in Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM), 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.
- [12] H. Steck, *Calibrated Recommendations*, in Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18), 2018, pp. 154–162, doi: 10.1145/3240323.3240372.
- [13] Y. A. Malkov and D. A. Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018, doi: 10.1109/TPAMI.2018.2889473.
- [14] J. G. Carbonell and J. Goldstein, *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*, in Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98), 1998, pp. 335–336, doi: 10.1145/290941.291025.
- [15] N. N. Anh, *Balancing Relevancy and Diversity with Two-Stage Maximal Marginal Relevance (2-Stage MMR) for Fashion Recommendation*, Internship 1 Final Report, Ho Chi Minh City University of Technology, 2026.
- [16] Google, *Firebase Documentation: Authentication, Firestore, and Cloud Functions*, 2024. <https://firebase.google.com/docs>.
- [17] Google, *Cloud Run: Managed Container Platform Documentation*, 2024. <https://cloud.google.com/run/docs>.
- [18] LiveKit Inc., *LiveKit Documentation: Real-Time Voice and Video Infrastructure*, 2024. <https://docs.livekit.io/>.
- [19] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, *Robust Speech Recognition via Large-Scale Weak Supervision*, arXiv preprint arXiv:2212.04356, 2022.

Table 2: Firestore collections relevant to the AI subsystem.

Collection	Purpose and key fields
users	Profile and roles. Fields: uid, email, display_name, roles[], phone_number, selected_job_category. Used to prime the tutor with “who is talking to me”.
courses	Course metadata and the AI-generated knowledge map. Fields: title, description, knowledgeMap (DAG of lesson nodes), createdBy.
lessons	Lesson content and per-user state.
conversations	Per-session metadata. Fields: member_uids[], course_id, modality, model, sessions[], feedback, last_active_at. Each conversation may aggregate one or more Gemini Live sessions.
messages	Per-turn transcript. Fields: conversationId, authorId, role (user/assistant), content, createdAt.
gemini_sessions	Per-WebSocket-session telemetry. Fields: uid, startTime, endTime, tokenUsage (input/output/total), messageCount, status.
tasks	Cloud Tasks records for asynchronous knowledge-map and lesson generation. Fields: type, status, payload, result.
whitelist_accounts	Pre-launch access control. Used by the auth middleware.

Table 3: Operational metrics, definitions and target thresholds.

Metric	Definition	Target
Round-trip latency (p50)	Median time between a client message and the first byte of the model response, measured at the proxy.	< 250 ms
Round-trip latency (p95)	95 th percentile of the above.	< 500 ms
Concurrent sessions per instance	Active WebSocket sessions an autoscaled Cloud Run instance can hold without queuing.	$\geq 1,000$
Message failure rate	Fraction of frames that fail to deliver in either direction.	< 1%
Connection failure rate	Failed WebSocket upgrades per minute.	< 10/min
Memory utilisation	Steady-state memory per instance.	< 80%

Table 4: Behavioural metrics tracked through Mixpanel and Firestore.

Metric	Definition
Prompt engagement rate	Homepage visitors who submit a skill query / total visitors.
Assessment completion rate	Users who finish the initial skill assessment / users who start.
Assistant interaction rate	Sessions in which the user exchanges ≥ 3 messages with the tutor / total session starts.
Sign-up conversion	Anonymous users who sign in within 24 h of their first session / anonymous users with at least one session.
7-day retention	Authenticated users who return within 7 days of sign-up.
Feedback rating distribution	Distribution of thumbs-up vs thumbs-down on conversation feedback.
Lesson completion rate	Lessons whose <code>is_completed</code> flips to <code>true</code> per session start.